Spring Boot Essentials

3-Day Workshop: Building Scalable APIs

Day 4

Module 8: Spring Boot & DB integration Basics

Dependencies

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
<!-- Spring Data JPA -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

<!-- H2 In-Memory Database -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
<!-- Validation -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
```

Configuring Spring Boot Data Source and JPA Settings

Datasource Configuration

Datasource URL, username, and password ensure reliable PostgreSQL database connectivity in Spring Boot.

JPA and Hibernate Settings

Hibernate ddl-auto validation and PostgreSQL dialect settings maintain schema integrity and SQL optimization.

SQL Debugging Features

Show-sql and format_sql properties enhance debugging by displaying readable SQL queries during execution.

Configuring Spring Boot Data Source and JPA Settings

```
# H2 Database
spring.datasource.url=jdbc:h2:mem:cruddb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# H2 Console (access at http://localhost:8080/h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Modeling and Persisting Data with Spring Data JPA

Defining the Product Entity with Validation

Entity and Table Mapping

The Product class uses JPA annotations to map directly to the students table in the database.

Primary Key Generation

The id field uses @Id and @GeneratedValue with IDENTITY to auto-generate primary keys in PostgreSQL.

Field Validation Annotations

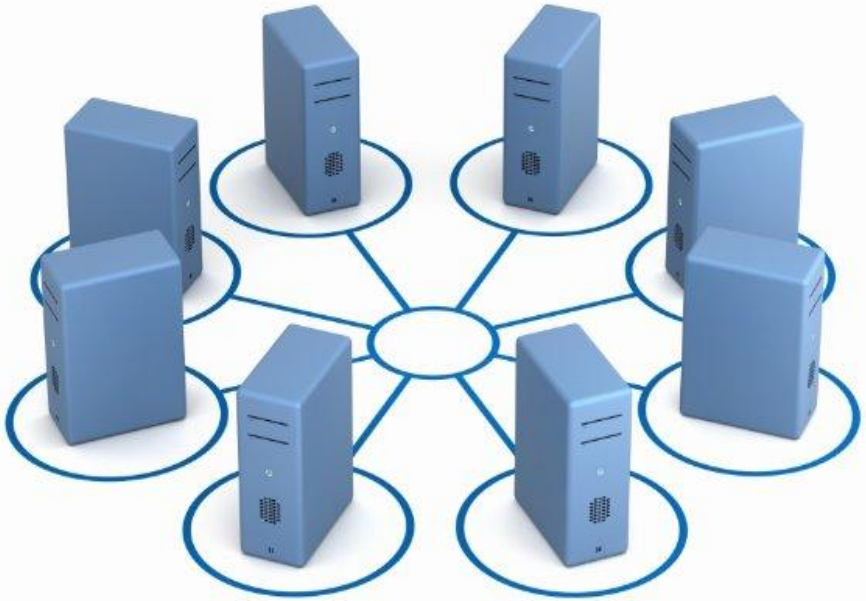
Validation annotations like @NotBlank ensure data integrity at application and API levels.

Sample Entity

```java
@Entity  no usages
@Table(name = "products")
public class Product {

    @Id  2 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")  3 usages
    private String name;

    private String description;  3 usages

    @NotNull(message = "Price is required")  3 usages
    @Positive(message = "Price must be positive")
    private Double price;

    private Integer quantity;  3 usages

    // Constructors
```

Building the Repository, Service, and Controller Layers

Implementing the Repository, Service, and REST Controller



Repository Layer

StudentRepository extends JpaRepository enabling CRUD operations and query derivation to simplify data access.

Service Layer Responsibilities

The service layer encapsulates business logic managing list, create, update, and delete operations with proper error handling.

REST Controller Endpoints

StudentController exposes RESTful endpoints using GET, POST, PUT, DELETE with validation and pagination support.

Clean Architecture and Workflow

Layered architecture promotes modularity, maintainability, and a clear workflow from model to testing.

Sample Repository

```
@Repository  no usages
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContainingIgnoreCase(String name);  no usages
}
```

Sample Service

```
@Service  2 usages
public class BookService {

    List<Book> books = new ArrayList<>();  3 usages

    public List<BookDTO> getAll() {  1 usage
        return books.stream() Stream<Book>
                .map( Book book -> {
                    BookDTO dto = new BookDTO();
                    dto.setTitle(book.getTitle());
                    dto.setAuthor(book.getAuthor());
                    return dto;
                }) Stream<BookDTO>
                .collect(Collectors.toList());
    }

    //insert
```

Sample Controller

```java
@RestController  no usages
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;  7 usages

    public ProductController(ProductService service) {  no usages
        this.service = service;
    }

    // POST /api/products
    @PostMapping  no usages
    public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
        return ResponseEntity.status(HttpStatus.CREATED).body(service.create(product));
    }

    // GET /api/products
    @GetMapping  no usages
    public ResponseEntity<List<Product>> getAll(
            @RequestParam(required = false) String name) {
        List<Product> products = (name != null)
                ? service.searchByName(name)
                : service.getAll();
        return ResponseEntity.ok(products);
    }
```

Exercise 8.10: Integrating DB

Hands-on Activity

Objective: Use the last Activity/Exercises integrate them using H2 db

Demonstrate by showing All request and SELECT * from “ÿour DB”

Group Activity

***M8GroupProject: Use the last API group activity and convert to SpringBoot and integrate to Postgres**